ASSIGNMENT 19

Dockerfile instruction

```
 => => exporting layers                                                                                                    4.1
 => => writing image sha256:33a2e53f84293c4819616e5b5b6c21b5980db866ac9f96c32aa9ca6221eca69b                               0.0
 => => naming to docker.io/library/firstimage                                                                              0.0
[root@ip-172-31-7-139 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED              SIZE
firstimage   latest    33a2e53f8429   About a minute ago   296MB
[root@ip-172-31-7-139 ec2-user]# docker run -itd -p 8080:80 --name firstcont firstimage:latest
717a65593d16da60e1efd1a857e0e8240e454ee03dfd1ccbba29b5cef6c7beef
[root@ip-172-31-7-139 ec2-user]# docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS          PORTS                                       NAMES
717a65593d16   firstimage:latest   "nginx -g 'daemon of…"   17 seconds ago   Up 16 seconds   0.0.0.0:8080->80/tcp, :::8080->80/tcp   firstcont
[root@ip-172-31-7-139 ec2-user]# ls
dockerfile  index.html
[root@ip-172-31-7-139 ec2-user]# cat dockerfile
FROM amazonlinux:2

# Install nginx
RUN yum update -y && \
    amazon-linux-extras install nginx1 -y && \
    yum clean all

COPY . /usr/share/nginx/html/
```

i-0f9bfdf728befdd65

PublicIPs: 13.223.174.232 PrivateIPs: 172.31.7.139

```
# Install nginx
RUN yum update -y && \
    amazon-linux-extras install nginx1 -y && \
    yum clean all

COPY . /usr/share/nginx/html/

# Expose port 80
EXPOSE 80

# Start nginx in foreground
CMD ["nginx", "-g", "daemon off;"]
[root@ip-172-31-7-139 ec2-user]# cat index.html
Here's a very simple HTML snippet where the content inside the <p> tag is "I'm Mourian":
<!DOCTYPE html>
<html>
<head>
    <title>Simple HTML Example</title>
</head>
<body>
    <p>I'm Mourian</p>
</body>
</html>

[root@ip-172-31-7-139 ec2-user]#
```

i-0f9bfdf728befdd65

PublicIPs: 13.223.174.232 PrivateIPs: 172.31.7.139

AWS Management Console | SecurityGroup | EC2 | us-east-1 | EC2 Instance Connect | us-east- | Repositories | mourianparthas

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-098fae8f569f6bbab&osUser=ec2-user&sshPort=22&addressFamily=...

Search [Alt+S] Ask Amazon Q | United States (N. Virginia) | Mourian (5104-9124-2048) | Mourian

```
 => => exporting layers                                                                                    0.0
 => => writing image sha256:43c7aaaf583dd0fb739cedb01698178cbb71127f696186694b98350073eaabc0               0.0
 => => naming to docker.io/library/secondimage                                                             0.0
[root@ip-172-31-15-106 ec2-user]# docker run -itd -p 8081:80 --name secondcont secondimage:latest
0e0df29ee148e97a0a92da492d9e68b5eb625454313dff610fa9493feb98d2f5
[root@ip-172-31-15-106 ec2-user]# docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS                                       NAMES
0e0df29ee148   secondimage:latest    "nginx -g 'daemon of…"   58 seconds ago   Up 57 seconds   0.0.0.0:8081->80/tcp, :::8081->80/tcp       secondcont
e8b4c2f489f7   firstimage:latest     "nginx -g 'daemon of…"   13 minutes ago   Up 13 minutes   0.0.0.0:8080->80/tcp, :::8080->80/tcp       firstcont
[root@ip-172-31-15-106 ec2-user]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
firstimage    latest    43c7aaaf583d   15 minutes ago   296MB
secondimage   latest    43c7aaaf583d   15 minutes ago   296MB
[root@ip-172-31-15-106 ec2-user]# docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Learn
 more at https://docs.docker.com/go/access-tokens/

Username: mourianparthasarathi
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
```

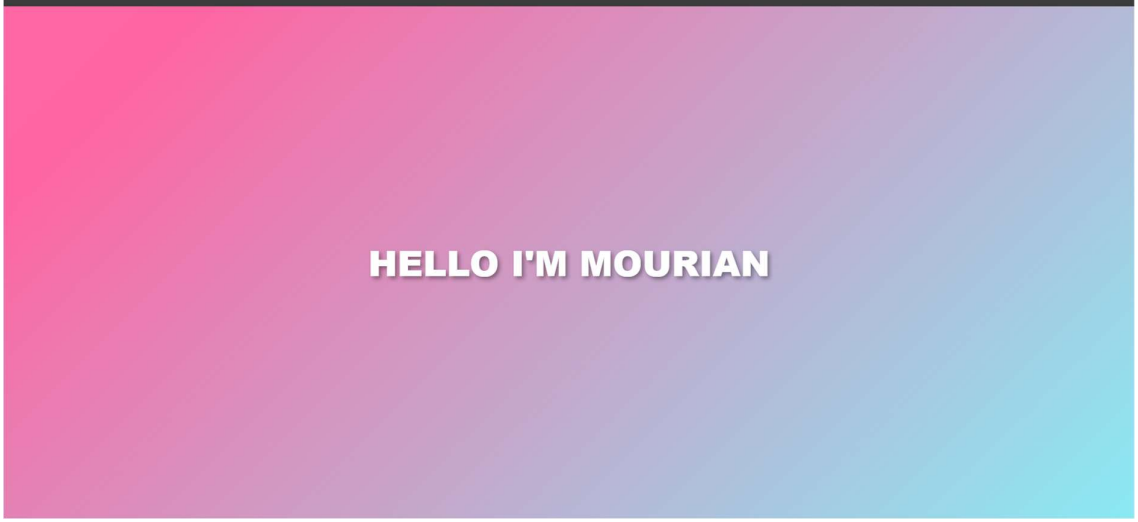
i-098fae8f569f6bbab

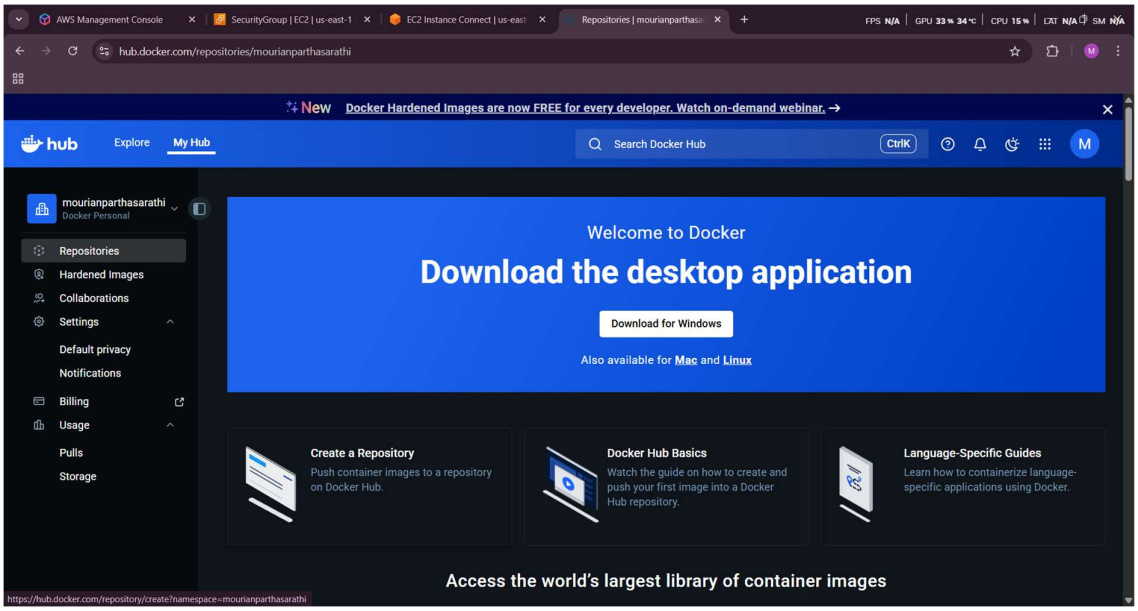
PublicIPs: 44.201.45.135 PrivateIPs: 172.31.15.106

CloudShell | Feedback | Console Mobile App | © 2026, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences

Hello Mourian

Not secure 44.201.45.135:8081





---

1. **yum install docker**
   This command is used to install Docker on a Linux system using the yum package manager. Docker allows us to create, run, and manage containers. Without installing Docker, containerization is not possible.

2. **systemctl start docker**
   This command starts the Docker service in the system. After starting the service, Docker commands like build, run, and push can be executed. If Docker service is not running, containers cannot be created or managed.

3. **nano dockerfile**
   This command opens the Dockerfile using the nano text editor. The Dockerfile contains instructions such as base image, copying files, and exposing ports, which are used to build a Docker image.

4. **nano index.html**
   This command creates or edits the index.html file. This file contains the web page content that will be displayed when the Docker container runs a web server.

5. **cat index.html**
   This command is used to display the contents of the index.html file on the terminal. It helps to verify whether the HTML content is written correctly.

6. **docker build -t firstimage .**
   This command builds a Docker image named firstimage using the Dockerfile present in the current directory. The dot (.) indicates the current directory is used as the build context.

7. **docker images**
   This command lists all Docker images available in the system. It shows image name, tag, image ID, and size, helping to confirm successful image creation.

8. **docker run -itd -p 8080:80 --name firstcont firstimage:latest**
   This command creates and runs a Docker container named firstcont from the firstimage. Port 8080 of the host is mapped to port 80 of the container, allowing web access through a browser.

9. **docker ps**
   This command displays all currently running Docker containers. It shows container ID, name, image, and status, helping to check whether the container is running successfully.

10. **nano index.html (again)**
    This command is used to modify the existing HTML file to change the web page content. After modification, a new image can be built to reflect the updated content.

11. **docker start**
    This command is used to start a stopped Docker container. It allows previously created containers to run again without creating a new one.

12. **docker build -t secondimage .**
    This command builds another Docker image named secondimage after modifying files like index.html. It helps to create a new version of the application.

13. **docker run -itd -p 8081:80 --name secondcont secondimage:latest**
    This command runs a second container named secondcont using the secondimage. It maps host port 8081 to container port 80, allowing multiple containers to run simultaneously.

14. **docker ps (again)**
    This command is used again to verify that both containers are running properly. It confirms container status and port mapping.

15. **docker images (again)**
    This command lists all Docker images including firstimage and secondimage. It helps to verify multiple image versions.

16. **docker login**
    This command logs in to Docker Hub using username and password. Authentication is required to push Docker images to an online repository.

17. **docker tag firstimage mourianparthasarathi/firstimage**
    This command tags the local Docker image with the Docker Hub repository name. Tagging is necessary before pushing the image to Docker Hub.

18. **docker tag firstimage:latest mourianparthasarathi/firstimage:latest**
    This command assigns the latest tag to the image for Docker Hub. It helps users pull the most recent version of the image easily.

19. **docker push**
    This command uploads the tagged Docker image from the local system to Docker Hub. Once pushed, the image can be downloaded and used anywhere.

20. **history**
    This command displays all previously executed commands in the terminal. It is useful for reviewing work done during the Docker session.